

Opening the Black Box

Mechanistic Interpretability for AI Agent Tool Selection Using Sparse Autoencoders

Hannes Hapke with **David Cardozo** · 575 Lab, Dataiku Inc.

ABOUT THE SPEAKER

Hi, I'm Hannes Hapke

Director, Engineering at Dataiku, 575 Lab

Google Developer Expert -- Machine Learning

Focus areas:

- Open Source Machine Learning
- Production ML systems & pipelines
- Generative AI and agentic systems

Co-author of four O'Reilly & Manning books, including:

- **Generative AI Design Patterns** (2025)
- **Machine Learning Production Systems** (2024)

Find me: hanneshapke.com • github.com/hanneshapke • linkedin.com/in/hanneshapke

Dataiku's Open Source Office



Advancing Responsible AI through open source — production-ready tools that give enterprises **transparency, control, and governance** to deploy AI with confidence.

Our Responsible AI work:

- **Trust** — no black boxes, no hidden behaviour
- **Control** — no vendor lock-in, no opaque dependencies
- **Accountability** — auditable, reproducible, community-reviewed

SIGNATURE PROJECTS

- **Kiji Inspector** — mechanistic interpretability for agent tool selection (*this talk*)
- **Kiji Privacy Proxy** — detect & redact sensitive data before LLM API calls

ALSO CONTRIBUTING TO

vLLM · scikit-learn (sponsor) · Cardinal (active learning) · CodeMirror

dataiku.com/open-source

The Problem: Opaque Agent Decisions

AI agents autonomously select tools (databases, web search, code execution, ...) based on natural language requests.

But **why**?

*"Find information about our company's API rate limits."
→ Internal docs search? Or public web search?*

Current approaches **fail** to provide mechanistic insight:

1. **Prompt engineering** -- reveals correlations, not causal mechanisms
2. **Behavioral testing** -- characterizes inputs/outputs, not internals
3. **Chain-of-thought** -- plausible narratives $\neq$ true computation

We need to look **inside the model**.

PART I

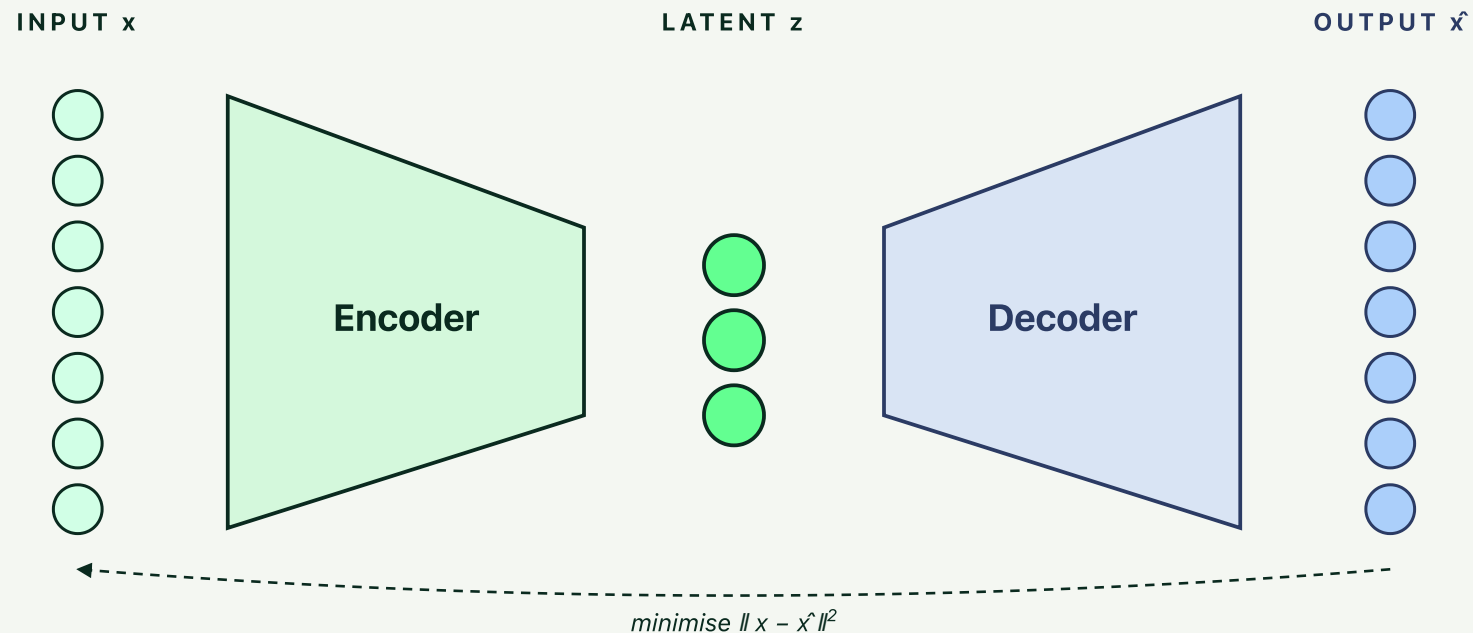
The Solution

LOOKING INSIDE WITH SPARSE AUTOENCODERS

How to Understand Models — Step 1

AUTOENCODERS

A **self-supervised** technique that learns new representations: compress an input through a bottleneck, then reconstruct it. What survives the bottleneck is what the model considers essential.

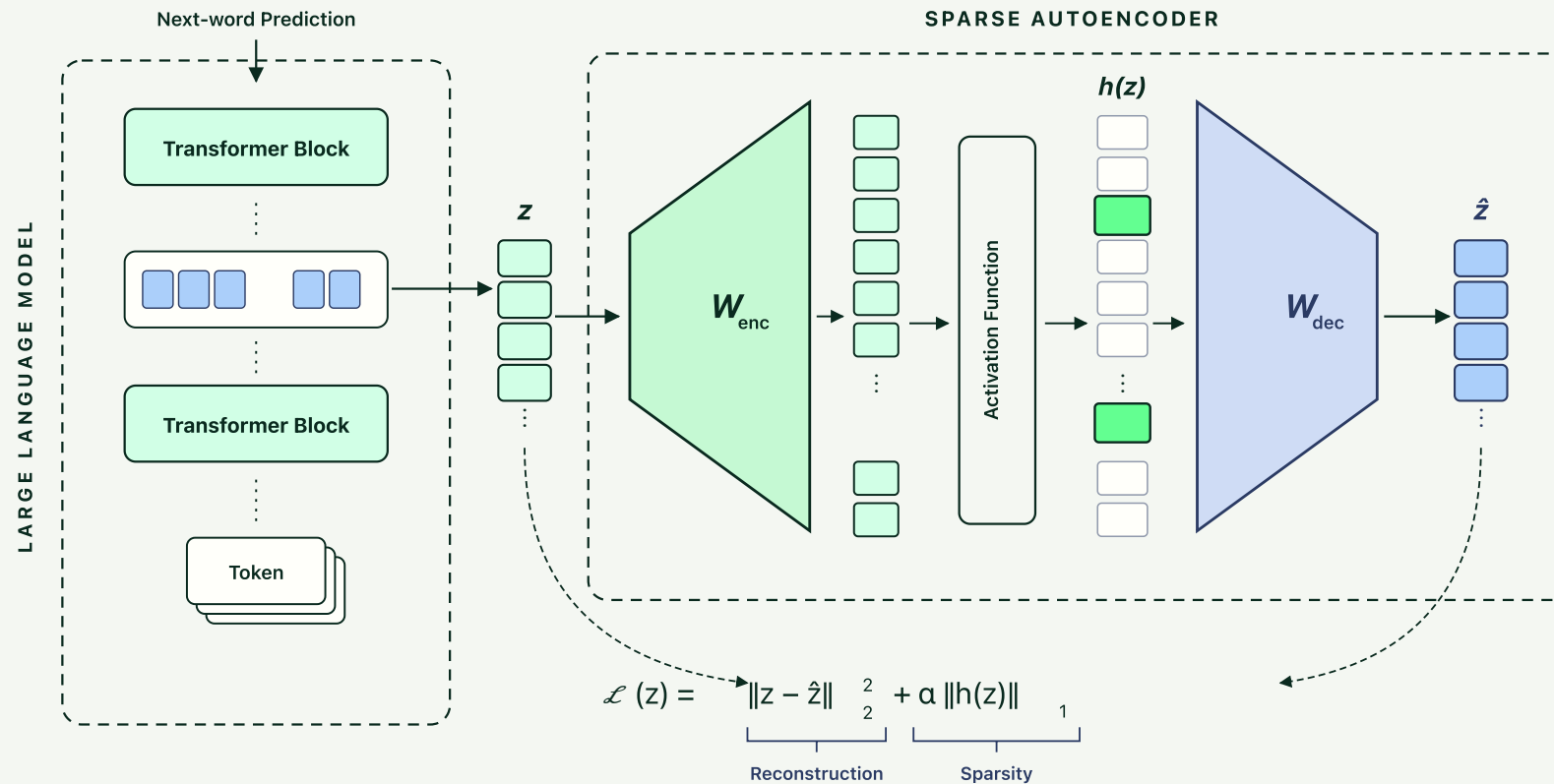


- **Well-understood** -- decades of theory and practice
- **Lossy by design** -- the bottleneck forces the model to *prioritise*

How to Understand Models — Step 2

SPARSE AUTOENCODERS (SAE)

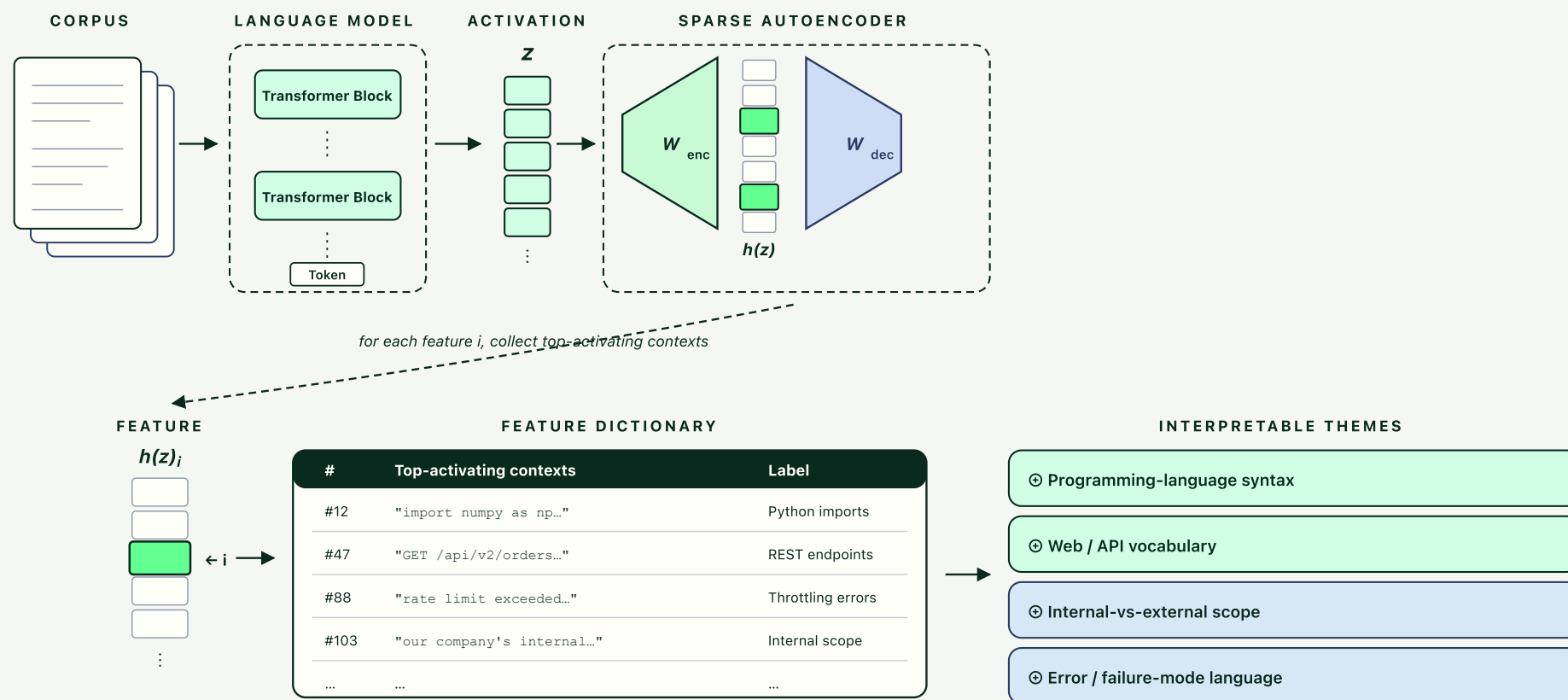
Flip the autoencoder on its head: instead of compressing, **expand** the latent space — but force fewer than **5%** of dimensions to fire on any given input.



How to Understand Models — Step 3

INTERPRETING WHAT THE FEATURES MEAN

A trained feature is just an index. To *label* it, collect the contexts where it fires most strongly — then let an LLM describe the pattern.



The Kiji Inspector

01

MODEL AGNOSTIC

Framework to train custom Sparse Autoencoders — works on **any open-source LLM**, not just the ones we tested.

02

END-TO-END TRAINING PIPELINE

No custom training setup needed. The project **generates the contrastive datasets** for you and runs the full extract → train → analyse loop.

03

PRODUCTION INFERENCE SUPPORT

vLLM patches, single-endpoint Docker containers, and PyPI packages — ready to drop into a live serving stack.

04

FULLY OPEN SOURCE

Pre-trained SAE models distributed via **Hugging Face, Docker Hub, and PyPI** — reuse without retraining.

What's Novel?

01

DECISION-TOKEN EXTRACTION

Capture activations at the *precise moment* of tool commitment — not averaged over the prompt.

02

PAIRS AS POST-HOC PROBES

The SAE learns the model's natural vocabulary **unsupervised**. Contrastive pairs are statistical probes only — never training signal.

03

TOKEN-LEVEL FUZZING EVALUATION

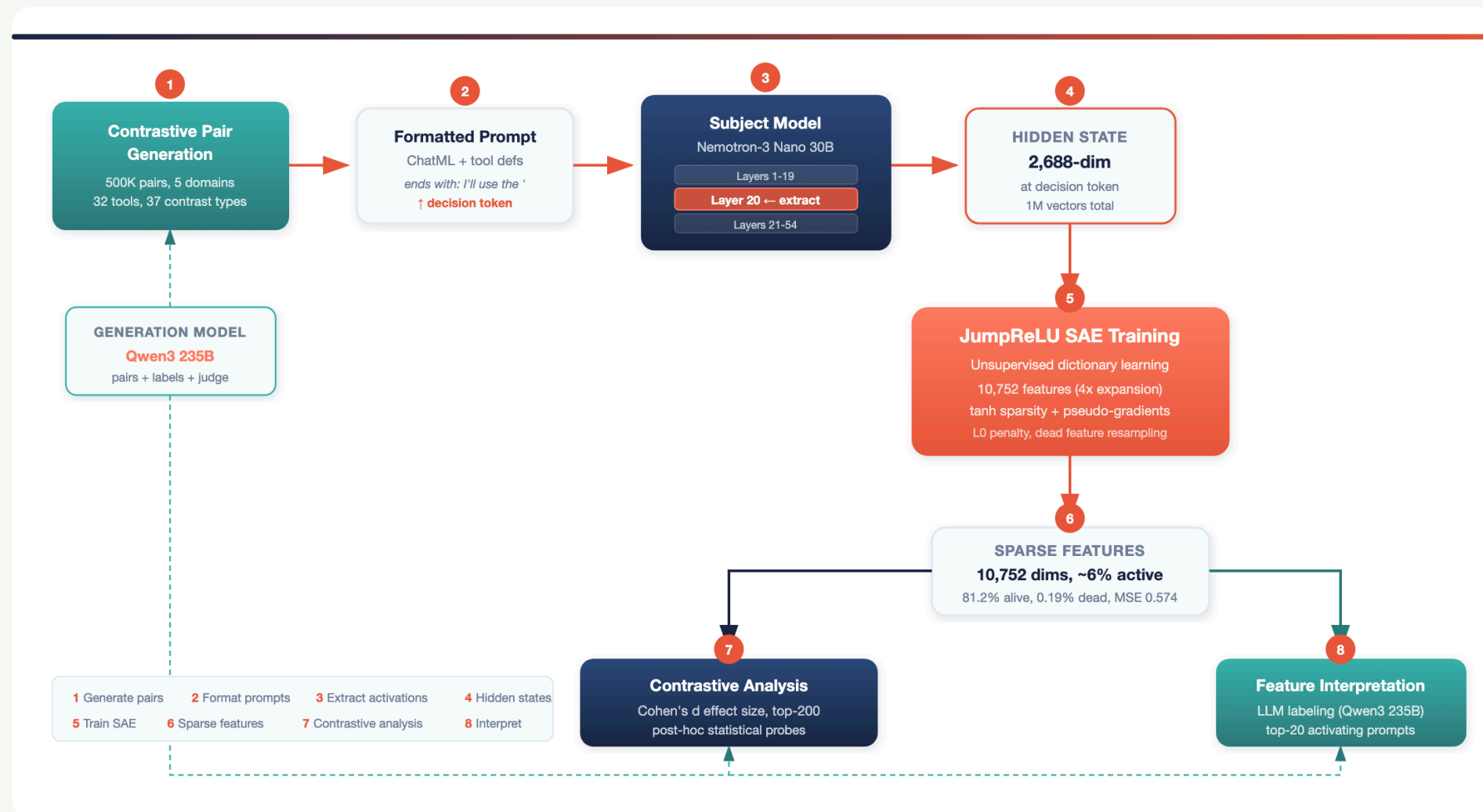
Adapted from Eleuther AI's autointerp — catches labels that are *"right for the wrong reasons."*

04

CAUSAL VALIDATION VIA ABLATION

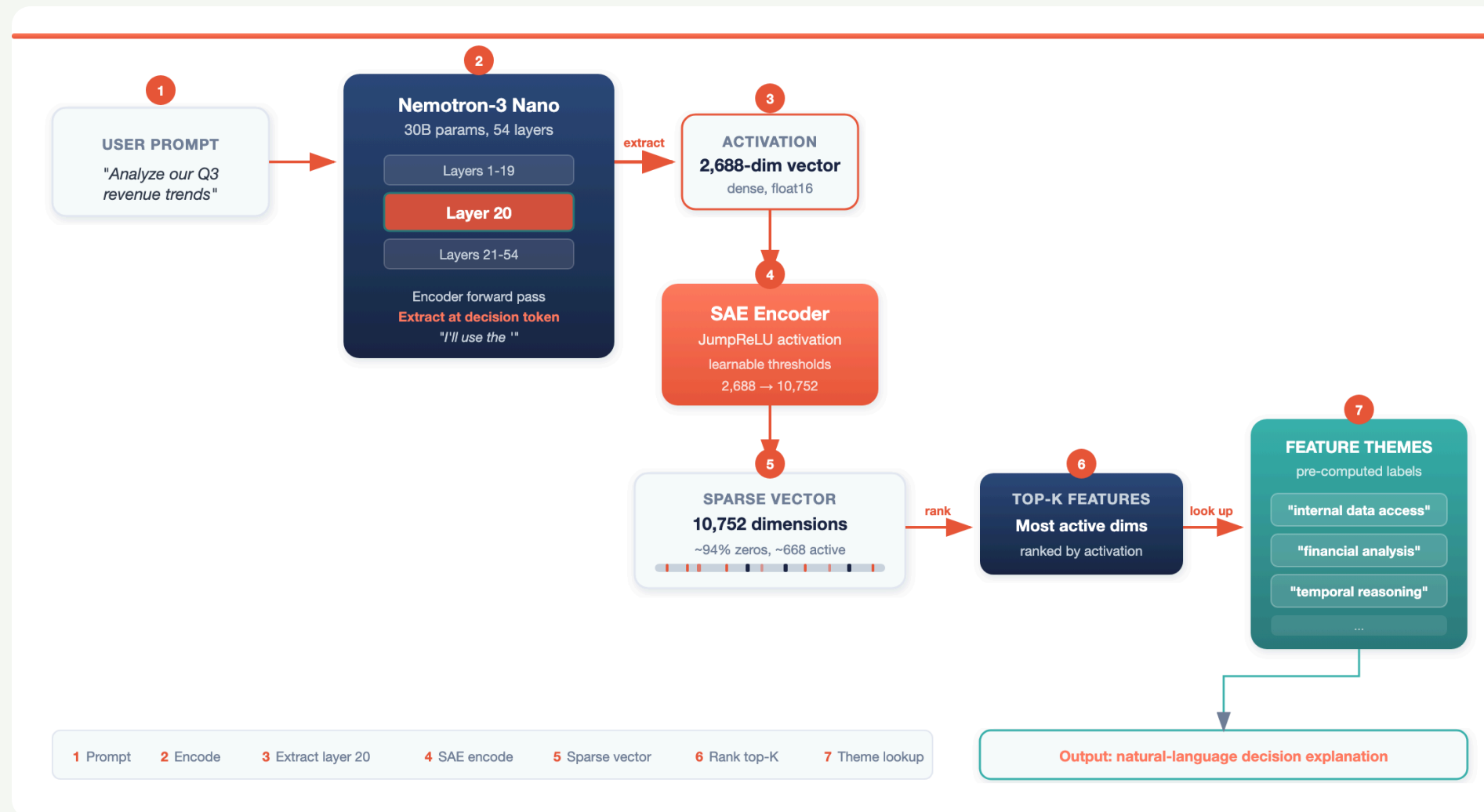
Zero a feature, measure the prediction flip — turns correlations into evidence.

Our Complete Training Pipeline



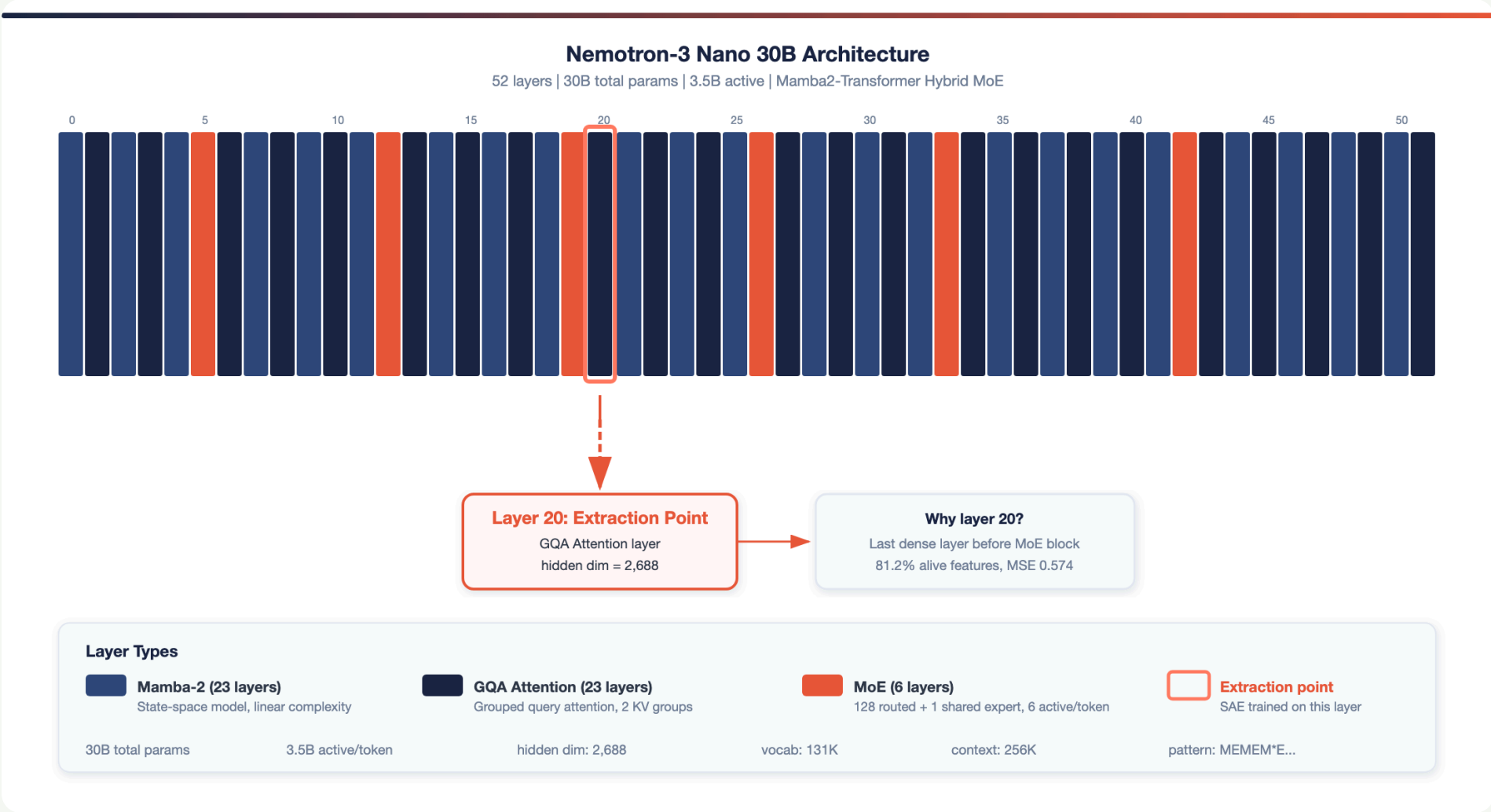
The SAE is trained unsupervised on the activations; contrastive pairs serve only as post-hoc statistical probes.

Our Inference Setup



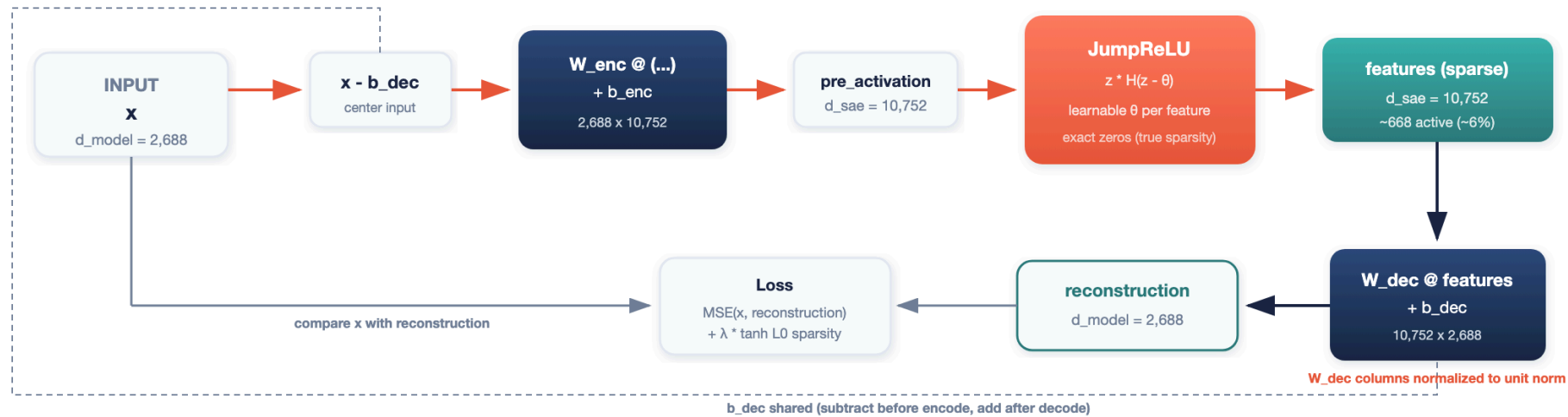
Seven steps from raw prompts to human-readable decision explanations.

Nemotron-3 Nano Architecture



Hybrid Mamba2-Transformer MoE, 52 layers, open weights &mdash. We extract activations at **layer 20** (GQA attention).

PyTorch SAE Model Architecture



Parameters

W_{enc} ($2,688 \times 10,752$)

W_{dec} ($10,752 \times 2,688$)

θ ($10,752$) thresholds

b_{enc} ($10,752$)

b_{dec} ($2,688$) shared

Total: $\sim 58\text{M}$ trainable parameters

Activation: JumpReLU with STE pseudo-gradients

Loss: $\text{MSE} + \lambda * \tanh\text{-smoothed L0}$

Encoder projects 2,688-dim input to 10,752 sparse features via JumpReLU with learnable per-feature thresholds.

Decision Token Extraction

Every formatted prompt ends with:

```
<|assistant|> I'll use the '
```

The hidden state at this final token is the **decision token** -- the model's internal state at the moment it commits to a tool name.

- Activations extracted at **layer 20** of Nemotron-3-Nano-30B (54-layer MoE) — *layer choice justified below*
- Hidden dimension: **2,688**
- Batched extraction with left-padding for alignment
- Dataset: **1,000,000** activation vectors (500K contrastive pairs)

Supported Language Models

Huggingface Model	Link	Kiji Inspector Sparse Autoencoder	Supported Layers
<code>google/gemma-3-27b-it</code>	model	575-lab/kiji-inspector-google-gemma-3-27b-it	10 20 31 41 50 58
<code>google/gemma-3-4b-it</code>	model	575-lab/kiji-inspector-google-gemma-3-4b-it	5 10 15 20 25
<code>google/gemma-3-1b-it</code>	model	575-lab/kiji-inspector-google-gemma-3-1b-it	5 10 15 20 25
<code>nvidia/Nemotron-3-Super-120B-A12B-FP8</code>	model	575-lab/kiji-inspector-NVIDIA-Nemotron-3-Super-120B-A12B-FP8	15 27 45 57 69 81
<code>nvidia/Nemotron-3-Super-120B-A12B-BF16</code>	model	575-lab/kiji-inspector-NVIDIA-Nemotron-3-Super-120B-A12B-BF16	15 27 45 57 69 81
<code>nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16</code>	model	575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-BF16	8 17 20 26 35 44
<code>nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-FP8</code>	model	575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-FP8	8 17 20 26 35 44
<code>nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-NVFP4</code>	model	575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-NVFP4	8 17 20 26 35 44
<code>google/gemma-4-31B-it</code>	model	575-lab/kiji-inspector-google-gemma-4-31B-it	10 20 31 41 50 58
<code>google/gemma-4-26B-A4B-it</code>	model	575-lab/kiji-inspector-google-gemma-4-26B-A4B-it	11 14 17 20 23

Hands-on: Extract a Layer's Activations

The Kiji Inspector builds on standard PyTorch primitives. Here's the extraction step in raw `transformers` — the same mechanism, no Kiji magic, on any HuggingFace causal LM.

```
pip install -U -q kiji-inspector transformers
```

Setup — Imports & Config

```
import torch
from transformers import AutoModelForCausalLM, AutoProcessor

LAYER_INDEX = 8
MODEL_ID = "google/gemma-4-E4B-it"
PROMPT = "My dishwasher is smelly, what is the first element I should review?"
```

Pick the **layer** to study and the **model** to study it on. Everything else flows from these three constants.

Load Model & Processor

```
processor = AutoProcessor.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
model.eval()
```

`bfloat16` halves memory; `device_map="auto"` lets HF Accelerate place layers across GPUs. `.eval()` disables dropout — we want deterministic activations.

Format the Prompt

```
messages = [{"role": "user", "content": [{"type": "text", "text": PROMPT}]]
prompt = processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True,
)
inputs = processor(text=prompt, return_tensors="pt")
inputs = {k: v.to(next(model.parameters()).device) for k, v in inputs.items()}
```

`add_generation_prompt=True` appends the assistant turn header — the model is now poised to *answer*. That final position is the **decision token** we just defined.

Capture Activations with a Forward Hook

```
captured = {}

def hook(_module, _inputs, output):
    hidden = output[0] if isinstance(output, tuple) else output
    captured["tensor"] = hidden.detach().cpu()

layer = model.model.language_model.layers[LAYER_INDEX]
handle = layer.register_forward_hook(hook)
```

PyTorch's `register_forward_hook` intercepts the layer's output mid-forward-pass. We `.detach()` to drop the autograd graph and `.cpu()` so we don't pin GPU memory.

Run Inference, Retrieve the Tensor

```
try:
    with torch.inference_mode():
        model(**inputs)
finally:
    handle.remove()

hidden_state = captured["tensor"]
```

`inference_mode()` is faster than `no_grad()` — it also skips view-tracking. The `try / finally` guarantees the hook is removed even on error. `hidden_state` is ready to feed into a trained SAE.

Now: Ask the Kiji Inspector What Fired

```
from kiji_inspector import SAE

sae, feature_descriptions = SAE.from_pretrained(
    base_model="google/gemma-4-E4B-it",
    layer=8,
)

# Activation for the first sequence, last token
last_token_act = hidden_state[0, -1, :]

# Describe the top features activating on this token
sae.describe(last_token_act, feature_descriptions)
```

`SAE.from_pretrained` pulls a layer-matched SAE *and* its labelled feature dictionary from the Hub. `describe()` returns the highest-activating features and their human-readable labels — the decision token, explained.

Output: Top Features on the Token

```
[(2341, 'unknown', 7.57),  
(14641,  
 {'label': 'Dishwasher Gasket Issues',  
  'description': 'Detects descriptions of dishwashers leaking from '  
                  'the door or bottom, often mentioning a worn, torn, '  
                  'or cracked door gasket.',  
  'confidence': 'high',  
  'mean_activation': 7.66,  
  'max_activation': 8.63,  
  'frac_nonzero': 1.0,  
  'top_examples': ['My 18-yr-old dishwasher is leaking from the door...',  
                  ...],  
  'bottom_examples': ['Find general best practices for API versioning...',  
                      ...]}},  
7.19),  
...]
```

A labelled feature (**#14641**) semantically matches the prompt — with full provenance (confidence, activation stats, witnessing contexts). Feature **#2341** fired too but autointerp hasn't named it. The trailing float in each tuple is the

Contrastive Pair Design

Pairs share the same *intent* but require *different tools*:

Shared Intent	Anchor (tool A)	Contrast (tool B)
Resolve password issue	"How do I reset my password?" → <code>knowledge_base</code>	"I tried resetting 3 times but the email never arrives" → <code>ticket_lookup</code>
Evaluate energy stocks	"Which companies invest in renewables?" → <code>financial_analysis</code>	"Which stocks trade below book value?" → <code>market_data_lookup</code>
Check product version	"What is the latest version?" → <code>file_read</code>	"Set the version to v3.2.1" → <code>file_write</code>

5 domains, 32 tools, 37 contrast types.

JumpReLU SAE Architecture

JumpReLU = ReLU with a learnable per-feature threshold — gives exact zeros but stays trainable via tanh pseudo-gradients (Rajamanoharan et al., 2024).

Encoder:

$$f_i(\mathbf{x}) = \pi_i(\mathbf{x}) \cdot H(\pi_i(\mathbf{x}) - \theta_i)$$

where $\pi_i(\mathbf{x}) = [W_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{dec}}) + \mathbf{b}_{\text{enc}}]_i$ and θ_i is a learnable threshold.

Key properties:

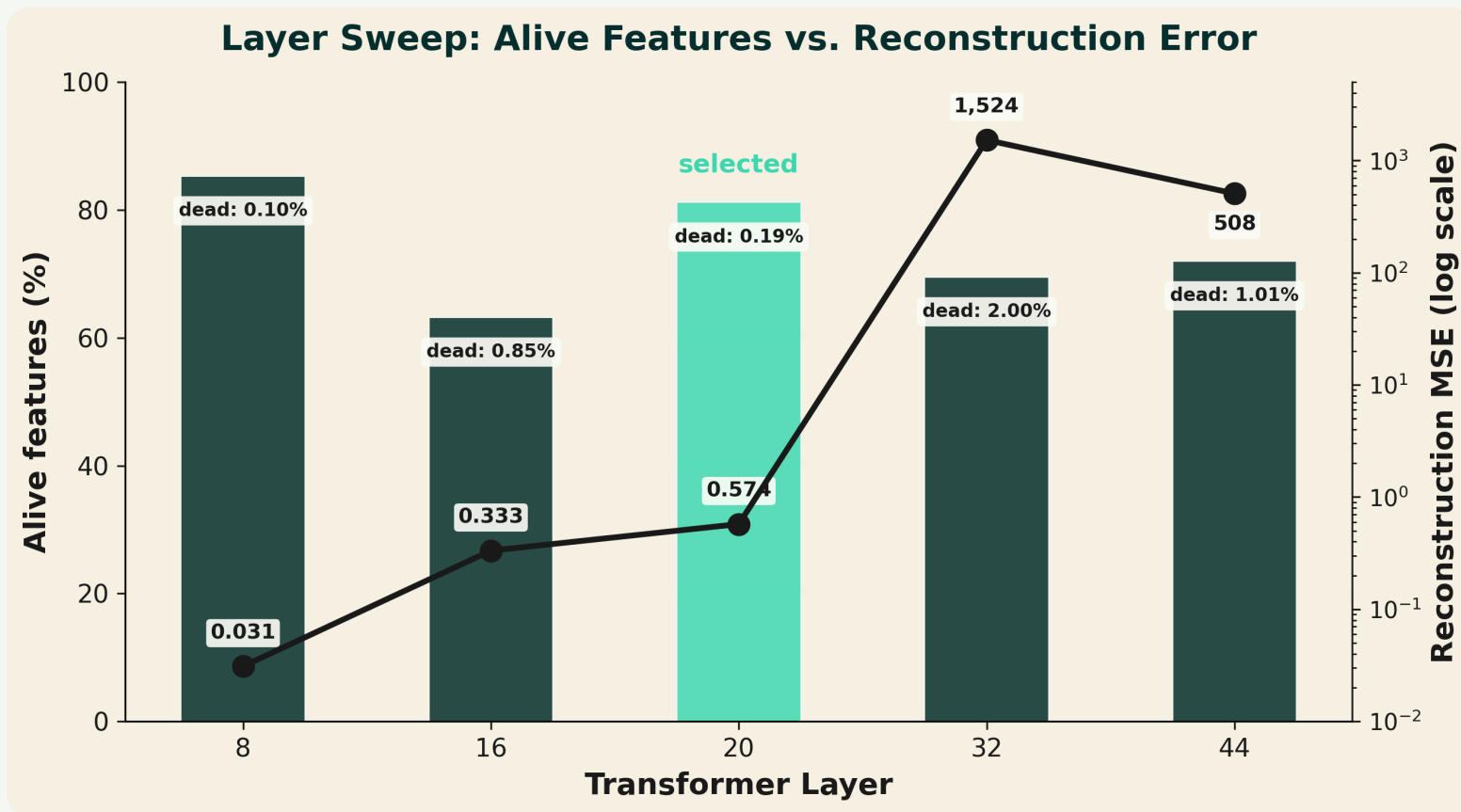
- **Exact sparsity** -- Heaviside step function H produces true zeros
- **Smooth training** -- tanh approximation for gradient flow:

$$\hat{\mathcal{L}}_{\text{sparse}} = \sum_{i=1}^M \text{ReLU} \left(\tanh \left(\frac{\pi_i - \theta_i}{\varepsilon} \right) \right)$$

- **Pseudo-gradients** via rectangular kernel density estimator

Dictionary size $M = 16,384$ ($4 \times$ hidden dim).

Why Layer 20?



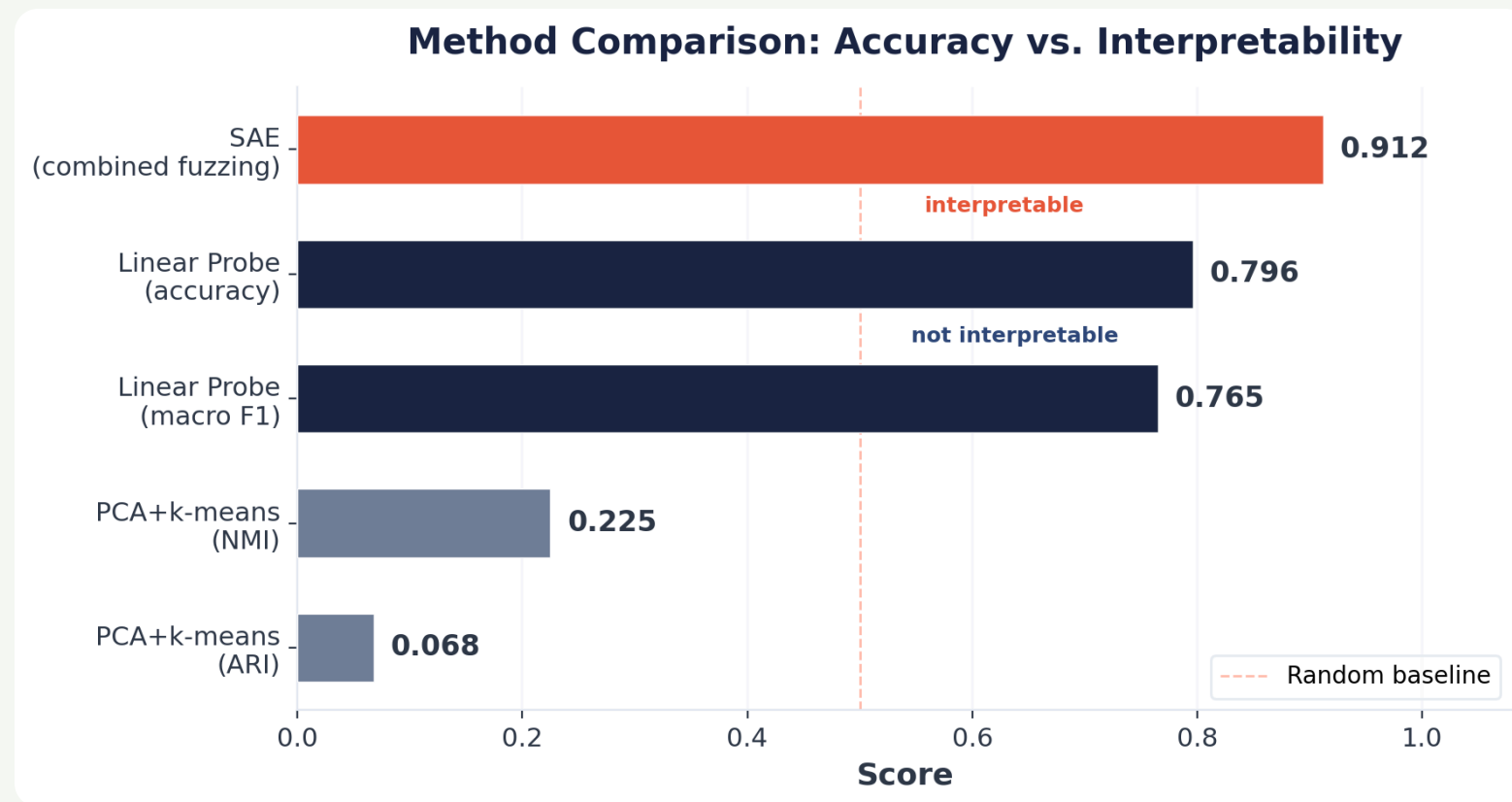
- Layers 8/16: low MSE but *pre-decision* representations
- **Layer 20**: best alive %, lowest dead %, MSE < 1.0
- Layers 32+: MoE expert routing → 500x+ higher MSE

SAE Feature Health (Layer 20, Full Dataset)

Metric	Value
Total features	16,384
Alive features (>0.1% firing)	81.2% [80.6, 81.8]
Dead features (0% firing)	0.19% [0.13, 0.27]
LO (active features per input)	668 ($\approx$ 4.1% density)
Reconstruction MSE	0.574

The SAE efficiently uses its capacity: sparse encoding with high feature utilization.

Baselines: Why Not Just a Probe?



- Linear probe confirms tool identity is *linearly encoded* (79.6% across 32 classes) -- but provides *no interpretability*
- PCA + k-means fails entirely -- tool signal is not dominant variance
- The SAE bridges this gap: **interpretable** *and* **causally testable** features

Token-Level Fuzzing Evaluation

Autointerp (EleutherAI) is the standard recipe for labelling SAE features at scale: an LLM proposes a label from top-activating contexts, a second LLM judges whether the label predicts activation. We tighten the test.

Standard evaluation: "Does this label predict which *prompts* activate the feature?"

Our evaluation: "Does this label predict which *tokens* activate the feature?"

Protocol:

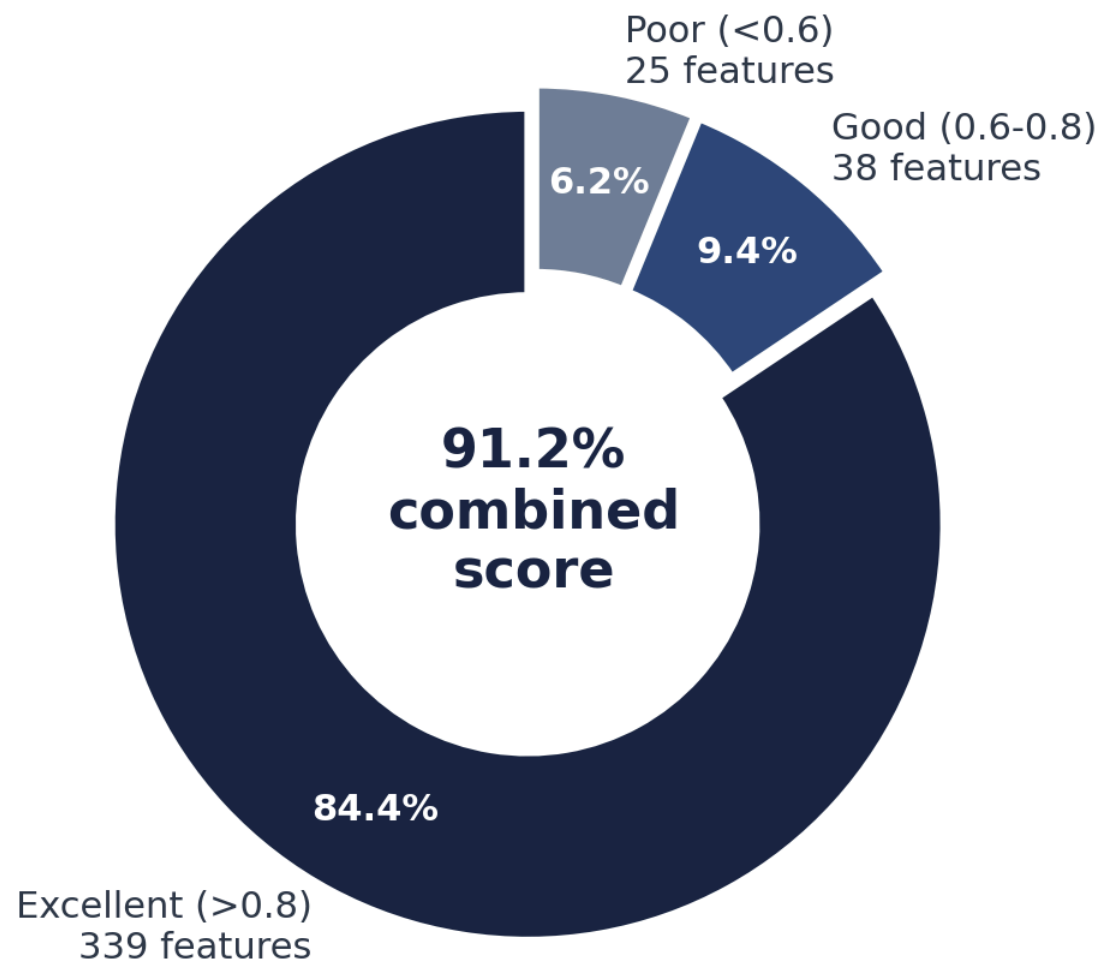
1. Extract per-token activations across entire prompt
2. Highlight top-K tokens in user request span
3. A/B test: LLM judge picks which highlighted text matches the label
4. Randomized order to prevent position bias

Combined score = $0.7 \cdot \text{acc}_{\text{token}} + 0.3 \cdot \text{acc}_{\text{prompt}}$

Token-level gets higher weight because it tests the *actual mechanism*.

Fuzzing Results: Features Are Interpretable

Feature Label Quality (Token-Level Fuzzing)



PART II

The Causality Test

From Correlation to Causal Evidence

Feature Ablation: Experimental Design

Question: Are contrastive features *causally necessary* for tool selection, or merely correlated?

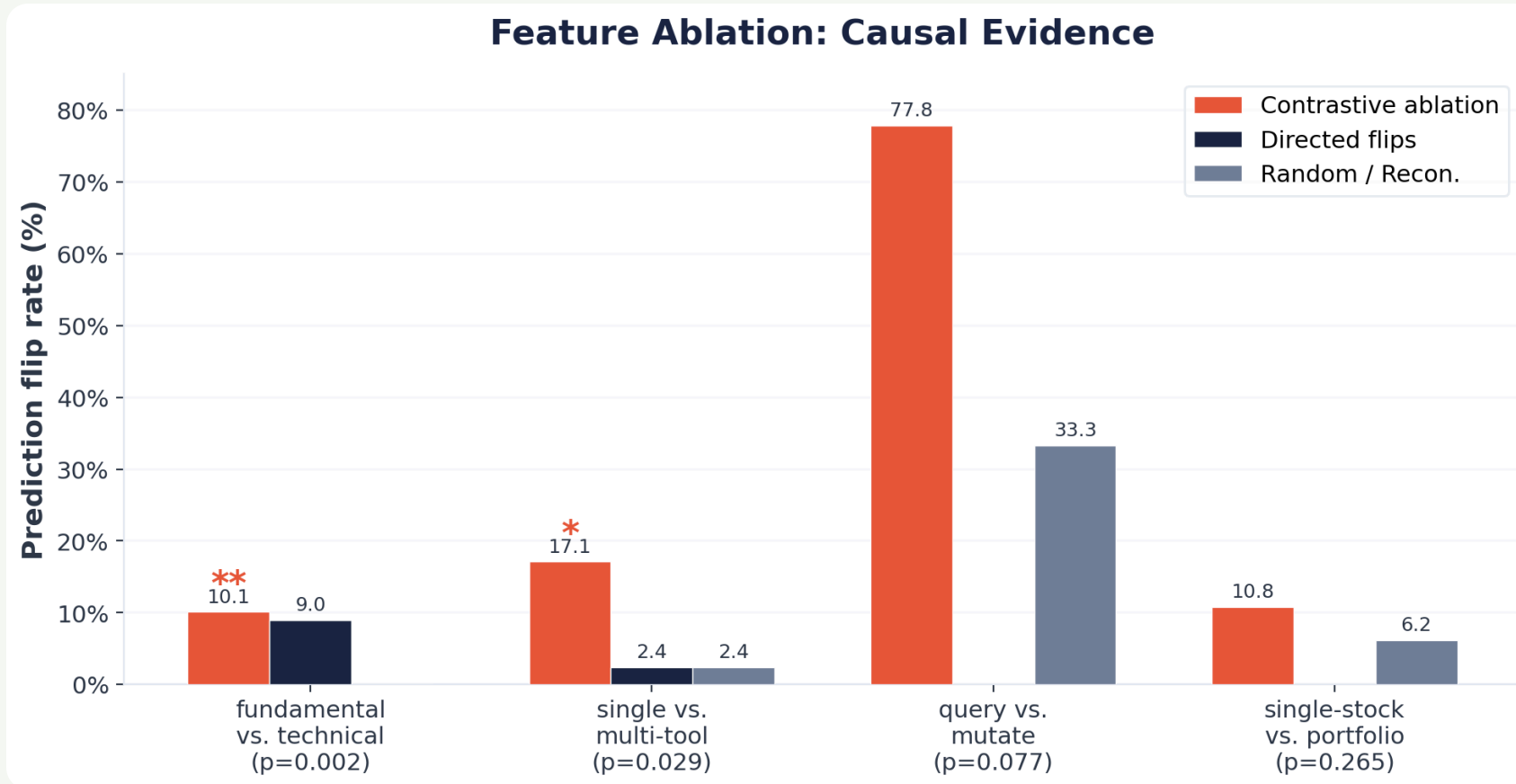
Method:

1. Intercept residual stream at layer 20
2. Encode through trained SAE
3. **Zero out top-10 contrastive features**
4. Decode back into residual stream
5. Measure: does the model's tool prediction *flip*?

Controls:

- **Random ablation:** zero 10 random non-contrastive features
- **Reconstruction-only:** SAE encode → decode with *no* features zeroed (measures round-trip distortion)

Ablation Results: Causal Evidence



Aggregate (23 types): 16.1% contrastive vs. 13.0% reconstruction-only

Why This Matters: Interpreting the Ablation

Fundamental vs. Technical Analysis ($p = 0.002$):

- Zeroing 10 contrastive features flips **10.1%** of predictions
- 9.0% flip *toward the contrast tool* (directed change)
- Random ablation: **0%** flips. Reconstruction-only: **0%** flips
- These features are *causally necessary* for this distinction

Critical control insight:

Random ablation flip rate equals SAE round-trip distortion across all 23 contrast types. Removing 10 random features from ~668 active adds no disruption beyond the encode-decode cycle itself.

This validates the design: ablation effects are due to *specific features*, not general signal degradation.

The Spectrum of Causal Involvement

Not all decisions rely on sparse feature circuits. Across 23 contrast types we see a spectrum:

- **Sparse circuits** -- a small minority (e.g. fundamental/technical, single/multi-tool) concentrate causal signal in a handful of identifiable features; ablating 10 contrastive features flips up to 10.1% of predictions
- **Distributed encodings** -- many contrast types (e.g. preventive/reactive maintenance) show 0% flips even with 10 features ablated, robust to any 10-feature subset
- **Intermediate** -- the remainder show detectable but not statistically dominant feature involvement

This reveals a heterogeneous landscape:

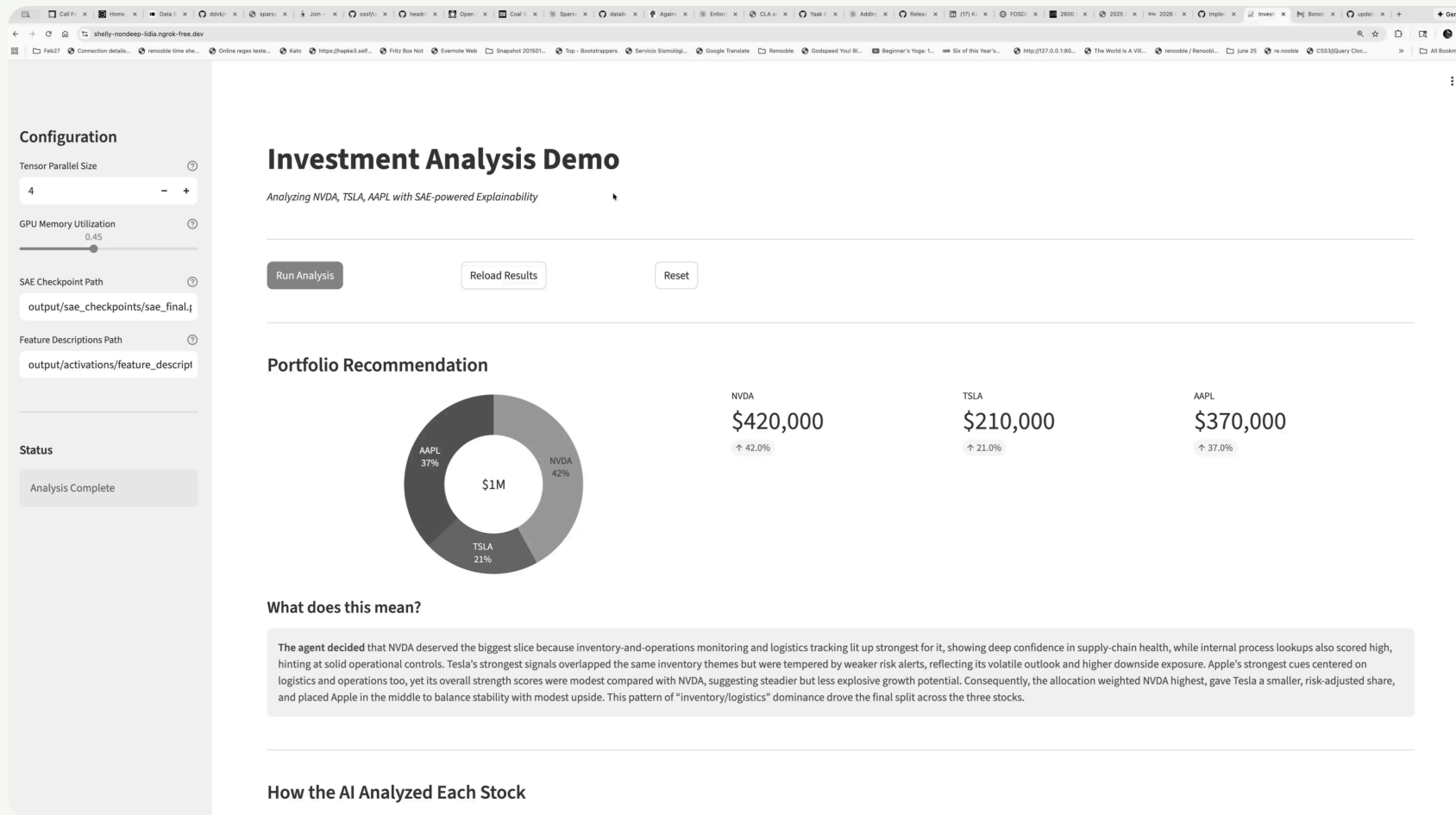
Some tool-selection decisions are governed by interpretable sparse circuits; others rely on distributed, redundant encodings.

Both findings are scientifically valuable.

Demo Time

KIJI INSPECTOR, LIVE

End-to-End Demo Application



Interactive system surfacing SAE-derived explanations alongside agent output -- translating internal feature activations into natural-language rationales.

Key Takeaways

01

SAEs discover interpretable decision features without supervision — 91.2% fuzzing accuracy

02

Token-level fuzzing catches labels *“right for the wrong reasons”* — a stricter test than prompt-level evaluation

03

Causal evidence via ablation: specific features are necessary for specific tool-selection decisions ($p = 0.002$)

04

The reconstruction-only baseline is essential — it separates genuine causal effects from SAE distortion artifacts

Limitations and Future Directions

CURRENT LIMITATIONS

- Training is compute-intensive
(235B generation model + 30B subject model)
- Access to model is required
- Synthetic contrastive pairs may miss real-world decision factors

FUTURE WORK

- Support for more open-source models like Qwen
- Multi-layer / circuit-level analysis
- Cross-model transfer (do tool-selection circuits generalize?)

Thank You

Open source: github.com/dataiku/kiji-inspector



Hannes Hapke — hannes.hapke@dataiku.com
David Cardozo — david.cardozo@dataiku.com
575 Lab, Dataiku Inc.